SQL COMMANDS:

DDL:

1.Create a table named EMPLOYEE with the following columns:

```
CREATE TABLE EMPLOYEE(
EMP_ID NUMBER(2) PRIMARY KEY,
EMP_NAME VARCHAR(20),
JOB VARCHAR(20),
SALARY NUMBER(6),
DEPTNO NUMBER(2)
);
```

2.Write a query to add a new column named HIREDATE with the DATE datatype to the existing EMPLOYEE table.

```
ALTER TABLE EMPLOYEE
ADD HIREDATE DATE;
```

3.Write a query to increase the size of the EMP_NAME column from VARCHAR2(20) to VARCHAR2(50) in the existing EMPLOYEE table.

```
ALTER TABLE EMPLOYEE
MODIFY EMP_NAME VARCHAR(50);
```

4.Write a query to remove the JOB column from the existing EMPLOYEE table.

```
ALTER TABLE EMPLOYEE
DROP COLUMN JOB;
```

5.Write a query to rename the EMPLOYEE table to STAFF.

```
RENAME EMPLOYEE TO STAFF;
```

6.Write a query to delete all records from the STAFF table while retaining its table structure.

```
TRUNCATE TABLE STAFF;
```

7.Write a query to permanently delete the STAFF table, including its structure and data.

```
DROP TABLE STAFF;
```

8.Create a table named PROJECT with the following requirements

```
CREATE TABLE PROJECT(
PROJECT_ID NUMBER(3) PRIMARY KEY,
PROJECT_NAME VARCHAR2(30) NOT NULL,
BUDGET NUMBER(8) DEFAULT 10000,
DEPTNO NUMBER(2)
);
```

9.Add START_DATE column AND Modify PROJECT_NAME size.

```
ALTER TABLE PROJECT
ADD START_DATE DATE;
```

```
ALTER TABLE PROJECT
MODIFY PROJECT_NAME VARCHAR(60);
```

10.Rename the column PROJECT_NAME to P_NAME THEN Rename the table PROJECT to PROJECT_DETAILS AND Remove the column START_DATE.

```
ALTER TABLE PROJECT
RENAME COLUMN PROJECT_NAME TO P_NAME;

ALTER TABLE PROJECT
ADD CONSTRAINT UK_PNAME UNIQUE;

RENAME PROJECT TO PROJECT_DETAILS;

ALTER TABLE PROJECT_DETAILS
DROP COLUMN START_DATE;
```

DML:

1.Consider the EMPLOYEE table above.

Task: Insert a new employee with:
- EMP_ID = 105
- NAME = 'Suresh'
- SALARY = 50000

```
INSERT INTO EMPLOYEE
VALUES(105,'Suresh',50000);
```

2.Change the salary of employee EMP_ID = 102 to 45000.

```
UPDATE EMPLOYEE
SET SALARY = 45000
WHERE EMP_ID = 102;
```

3.Delete the employee whose EMP_ID = 103.

```
DELETE FROM EMPLOYEE
WHERE EMP_ID = 103;
```

4.Change the salary to 60000 and the employee name to 'Suresh Kumar' for EMP_ID = 105.

```
UPDATE EMPLOYEE
SET SALARY = 60000,NAME = 'Suresh Kumar'
WHERE EMP_ID = 105;
```

5.Insert a new employee with:
- EMP_ID = 106
- NAME = 'Priya'
- SALARY = 40000

This time, explicitly mention the column names in your INSERT query.

```
INSERT INTO EMPLOYEE (EMP_ID,NAME,SALARY)
VALUES(106,'Priya',40000);
```

TCL:

1.Update the salary of employee EMP_ID = 101 to 55000 and permanently save the changes.

```
UPDATE EMPLOYEE
SET SALARY = 55000
WHERE EMP_ID = 101;

COMMIT;
```

2.Delete the employee whose EMP_ID = 105. Then undo the deletion using ROLLBACK.

```
DELETE FROM EMPLOYEE
WHERE EMP_ID = 105;

ROLLBACK;
```

3.Update employee 101's salary to 60000.Create a savepoint named S1.Update employee 102's salary to 70000.Roll back to savepoint S1.

```
UPDATE EMPLOYEE
SET SALARY = 60000
WHERE EMP_ID = 101;

SAVEPOINT S1;

UPDATE EMPLOYEE
SET SALARY = 70000
WHERE EMP_ID = 102;

ROLLBACK TO S1;
```

DCL:

1.You have an EMPLOYEE table and want to give user HARI permission to view employee records.

```
GRANT SELECT
ON EMPLOYEE
TO HARI;
```

2.User HARI currently has INSERT permission on the EMPLOYEE table.

```
REVOKE INSERT
ON EMPLOYEE
TO HARI;
```

3.Give user HARI both SELECT and UPDATE permissions on the EMPLOYEE table.

```
GRANT SELECT,UPDATE
ON EMPLOYEE
TO HARI;
```

DQL:

1.Display the NAME and SALARY of all employees whose salary is greater than 30000.

```
SELECT NAME,SALARY
FROM EMPLOYEE
WHERE SALARY > 30000;
```

2.Display all columns of employees whose salary is between 30000 and 50000 (inclusive).

```
SELECT * FROM EMPLOYEE
WHERE SALARY BETWEEN 30000 AND 50000;
```